

FILE XTRACTOR

Next-Generation Desktop Intelligent Content Search & Multimodal AI Vision Engine

Author / Developer: Antigravity & User	Platform: Windows 11 (x64)	Hardware Acceleration: NVIDIA RTX 4060 GPU (8GB VRAM)
Core Framework: Python 3.12 · PySide6 (Qt 6.8)	Deep Learning Engine: PyTorch 2.14 · Sentence-Transformers 6.1	Document & OCR: Tesseract 5.5 · Pillow 12.3 · pypdf 6.18

1. EXECUTIVE SUMMARY & VISION

FILE XTRACTOR is an enterprise-grade, privacy-first intelligent desktop asset discovery system that resolves the fundamental limitations of standard operating system search engines. Traditional OS tools index only static file names and basic metadata, failing when users search for content inside scanned documents, diagrams, stylized artwork, or when phrasing questions in conversational human language.

FILE XTRACTOR incorporates an entirely local, zero-cloud architecture combining **Full-Text Search (SQLite FTS5)**, **Offline Optical Character Recognition (Tesseract 5.5)**, **Deep Multimodal Neural Embeddings (OpenAI CLIP ViT-B-32)**, and an autonomous **Conversational Natural Language Agent**. All indexing, vectorization, and queries execute locally with zero external network dependencies, ensuring complete user privacy and instantaneous response times.

2. SYSTEM ARCHITECTURE & COMPONENT STRUCTURE

The application architecture is organized into four decoupled layers communicating through asynchronous signals and thread-safe boundaries:

Architectural Layer	Primary Modules	Responsibilities & Engineering Approach
Presentation Layer (UI/UX)	app.py, search_view.py, folder_manager.py,	Hardware-accelerated PySide6 interface. Implements 60fps downward-expanding

	<code>preview_pane.py</code> , <code>sidebar.py</code>	search capsule (56px to 132px), live QProgressBar per folder card, kinetic cipher scrambler, and pure monochrome styling (#151515 swatch).
AI & Vision Intelligence	<code>ai_agent.py</code> , <code>vision_search.py</code>	Natural language query parsing, academic acronym expansion (DBMS, DA 3, LA 1, OS), and local CLIP ViT-B-32 multimodal cosine similarity vector ranking.
OCR & Extraction Engine	<code>ocr.py</code> , <code>extractors.py</code> , <code>models.py</code>	Tesseract 5.5 binding with automated image enhancement (Lanczos upscaling, grayscale, 1.8x contrast). Extracts digital text from docx, pdf, txt, and page images. Video/ Movie heuristics classifier.
Persistence & Indexing Layer	<code>database.py</code> , <code>scanner.py</code>	SQLite 3 with Write-Ahead Logging (WAL), foreign keys, and FTS5 full-text virtual table. Dedicated QThread ScanWorker with strict per-file exception isolation.

3. MULTIMODAL VISION AI ENGINE (CLIP VIT-B-32)

Standard optical character recognition fails on graphic emblems, logos, and stylized horror/esports fonts (such as the *Crimson Crawlers* red spider logo) because they lack conventional Latin book typography. FILE XTRACTOR resolves this through a local **Vision Transformer Neural Network**.

Mathematical Formulation: Multimodal Joint Vector Space Image visual features and user text queries are projected into a common 512-dimensional normalized hypersphere:

- **Image Vector:** $v_{\text{image}} = \text{Normalize}(\text{VisionEncoder}(\text{Image})) \in \mathbb{R}^{512}$
- **Query Vector:** $v_{\text{query}} = \text{Normalize}(\text{TextEncoder}(\text{Query})) \in \mathbb{R}^{512}$
- **Relevance Metric:** $\text{CosineSimilarity} = v_{\text{query}} \cdot v_{\text{image}} = \sum v_{\text{query}, i} \times v_{\text{image}, i}$

When an image is indexed or searched, the engine evaluates cosine similarity against the query embedding. Scores above 0.20 indicate strong visual correlation. The raw similarity metric is mapped to a calibrated confidence percentage:

```
ConfidencePercentage = min(99, max(65, ((Similarity - 0.18) / 0.16) * 30 + 70))
```

Visual embeddings are cached by canonical file path and system modification timestamp (`st_mtime`), allowing subsequent visual searches across hundreds of local photos to complete in under **2 milliseconds**.

4. CONVERSATIONAL NATURAL LANGUAGE QUERY AGENT

Human users naturally query files using conversational language rather than rigid file paths.

The `AI-Agent` component orchestrates query normalization and multi-pass retrieval:

1. **Conversational Syntax Stripping:** Removes linguistic filler phrases ("*i need my*", "*where is the*", "*can you find*", "*sort of stuff*") via regex pipeline.
2. **Domain Acronym & Entity Expansion:** Automatically detects and cross-expands technical and university coursework terms:
 - DBMS ↔ *Database Management Systems, SQL, Database*
 - DA 3 / DA3 ↔ *Digital Assignment 3, Assignment 3, la 3, Lab Assessment 3*
 - OS ↔ *Operating Systems, BACSE106*
 - CN ↔ *Computer Networks* · DSA ↔ *Data Structures and Algorithms*
3. **Multi-Strategy Candidate Search:** Simultaneously executes filename pattern matching, full-text FTS5 BM25 retrieval, and multimodal CLIP image scoring, annotating results with high-contrast AI badges and explanation breadcrumbs.

5. HIGH-PRECISION OPTICAL CHARACTER RECOGNITION (OCR)

The OCR pipeline bridges the gap between digital text files and scanned media:

- **Engine Configuration:** Binds to local Tesseract 5.5 binary with Page Segmentation Mode 3 (Fully Automatic) and Neural LSTM Engine (OEM 1).
- **Image Preprocessing Pipeline:**

- Normalizes color channels to RGB.
- Bicubic / Lanczos upscaling if image dimensions are below 800px to ensure character strokes span at least 25 pixels.
- Converts to luminance grayscale (L).
- Applies high-pass contrast enhancement (factor: 1.8) to separate text from background textures.
- **Embedded Scanned PDF Processing:** Extracts image streams from PDF pages via `pypdf` when digital text is sparse or absent, processing up to 20 pages per document safely within thread memory constraints.

6. ZERO-CRASH CONCURRENCY & INDEXING ENGINE

Previous indexing attempts stalled when encountering corrupted, locked, or unsupported binary files. The re-engineered `ScanWorker` enforces strict per-file fault isolation:

- **Per-File Exception Boundary:** Every file operation is wrapped in an independent `try ... except Exception` block. A corrupted PDF or locked Windows system file logs a diagnostic record and increments a failure counter, while the scanner immediately advances to the next file without aborting.
- **Unconditional Indexing Registration:** Files lacking extractable text (e.g. video files, archives, non-text images) are formally committed as `indexing_status = 'indexed'`, guaranteeing accurate folder counters and search visibility.
- **Real-Time UI Telemetry:** Emits fine-grained progress updates (`current`, `total`, `filename`) forwarded directly to folder cards.

7. SMART VIDEO & MOVIE CLASSIFICATION

Video files are separated from generic media and classified into **Feature Films (Movies)** versus **Video Clips / Recordings**:

Classification	Detection Heuristics	UI Badging & Output
----------------	----------------------	---------------------

Movie / Feature Film	File size $\geq$ 350 MB, OR filename matches release resolution (1080p, 720p, 4K), rip source (BluRay, WEBRip, HDRip), codec (x264, x265, HEVC), release group (YTS), or 4-digit release year (19xx/20xx).	[ MOVIE (1080P)] badge with glowing white border, classified under dedicated Video category.
Standard Video Clip	Shorter duration, screen recordings, mobile captures, WhatsApp videos, or tutorials under 350 MB without release tags.	[▶ VIDEO] badge, displays exact resolution and human-readable file size in MB.

8. DATABASE SCHEMA & PERSISTENCE SPECIFICATION

Persistence is managed via SQLite 3 configured with Write-Ahead Logging (`PRAGMA journal_mode = WAL`):

- `indexed_folders`: Stores unique folder paths, initial registration timestamps, and last scanned timestamps.
- `files`: Comprehensive file catalog tracking `id`, `folder_id`, `filename`, `canonical path`, `extension`, `file_type`, `size_bytes`, `timestamps`, `extracted_text`, `ocr_text`, and `indexing_status`.
- `file_search` (SQLite FTS5): Virtual full-text search table indexing `file_id`, `filename`, and combined document text with BM25 ranking algorithm and automated snippet extraction.

9. REAL-WORLD EMPIRICAL VERIFICATION RESULTS

The system was validated against actual user files on the host machine:

User Search Query	Target File Identified	AI Confidence / Score	Match Explanation
"crimson crawlers"	final_1.png	99% Confidence (0.344)	CLIP Multimodal Vision Match
"red spider logo"	final_1.png	89% Confidence (0.286)	CLIP Multimodal Vision Match
"i need my DBMS DA 3"	25BCE5146 la 3 .pdf	100% Match	Acronym expansion: DA 3 ↔ la 3
"where is my operating systems pdf"	BACSE106_OPERATING-SYSTEMS...pdf	100% Match	Natural language query normalizer
"flow diagram for how to start a startup"	WhatsApp Image 2026-09-17...jpeg	91% Confidence (0.293)	Diagram visual structure + OCR text

10. CONCLUSION

FILE XTRACTOR establishes a new standard for local desktop content discovery. By combining deep learning vision models, resilient multithreaded scanning, and natural language query translation with an elegant, responsive Qt6 desktop interface, the application delivers instant, accurate retrieval of any file, photo, or document without ever compromising user privacy.